Contents

1. Introduction **3**
- 1.0.1. Related work and our contribution . . . 4
- 1.1. Linked repositories . . . 5

2. Auditing the Belgian Federal Elections **5**
- 2.1. Context of the Audit . . . 5
- 2.2. The election manifest . . . 6
- 2.3. Auditing D'Hondt Elections . . . 6
  - 2.3.1. D'Hondt assertions . . . 7
- 2.4. Adding a threshold . . . 8
- 2.5. Summary: checking the election outcome . . . 8
  - 2.5.1. Assigning all seats . . . 9

3. Which assertions should be checked? **10**
- 3.1. Complete assertion sets . . . 10
- 3.2. Relaxed assertion sets . . . 10
  - 3.2.1. Motivating example . . . 10
  - 3.2.2. Verifying relaxed assertion sets . . . 11

4. Deriving relaxed assertion sets **13**
- 4.1. Equivocating over whether a party failed the threshold . . . 13
  - 4.1.1. Algorithm for relaxing threshold assertions . . . 15
  - 4.1.2. Algorithm for threshold uncertainty . . . 15
- 4.2. Relaxing in the case that a DH assertion is infeasible . . . 17
  - 4.2.1. Relaxing multiple DH comparisons . . . 18
  - 4.2.2. DH-relaxing algorithm . . . 18
- 4.3. Relaxing threshold and DH assertions . . . 19
  - 4.3.1. Complete algorithm . . . 19
  - 4.3.2. Summary/implications . . . 20
  - 4.3.3. Summary of Assertions . . . 20

A. Defining assorters for DH and AT assertions. **21**
- A.1. D'Hondt assorters . . . 21
- A.2. Threshold assorters . . . 21

B. Estimating which disjunct requires fewer samples **22**
- B.1. Check margins . . . 23
- B.2. Compare expectations based on ALPHA martingales . . . 23
- B.3. Compare expectations based on RAIRE (or) AWAIRE) . . . 24

Feasible Risk Limiting Audits for Belgium

November 6, 2026

1. Introduction

The Belgian federal elections select 150 members for the Chamber of Representatives (≈ lower house) of the Federal Parliament, following a process that is described in the "Code électoral".[1] These 150 Representatives come from 11 electoral districts, that run 11 independent elections, each for a fixed number of representatives that currently ranges from 4 to 24 as a function of the population of the district (these numbers are adjusted every ten years, following a process defined at Art. 63 of the Belgian constitution).

Each district proposes a ballot to its voters, on which candidates are grouped into parties. In order to vote, a voter can either pick a party or approve as many candidates as they want within a single party (Art. 144 of the "Code électoral"). An example of ballot is visible in Fig. 1. Approving candidates across multiple parties makes an invalid ballot.

The tallying operations are performed independently in each electoral district. Each counting office reports the number of votes that each party received, which are added at the district level. The only parties that can receive a seat are those that obtained the minimum electoral threshold of 5% of the total number of valid votes (Art. 165bis), that is, ballots on which a preference is expressed for one single party (either for the party or by approving candidates on the party) — blank votes are not considered valid (Art. 157).

The seats that are available for the district are then attributed according to the D'Hondt method: the totals of each party that is above the electoral threshold are divided by 1, 2, 3, etc. up to the number of candidates in the party, resulting in lists of quotients, which are merged in descending order. If S seats are to be attributed in a district, then each party receives a number of seats that equals the number of appearance of its quotients within the top S quotients of the merged list (Art. 167). We illustrate this process in Fig. 2.

In case of equality of quotients, the seat is attributed to the party that has the highest total number of votes. If that is not sufficient to solve the tie, then the candidates who would get the seats are compared: the seat goes to the party of the candidate who has

[1] `https://elections.fgov.be/legislation/lois`, last updated on Oct. 27, 2023.

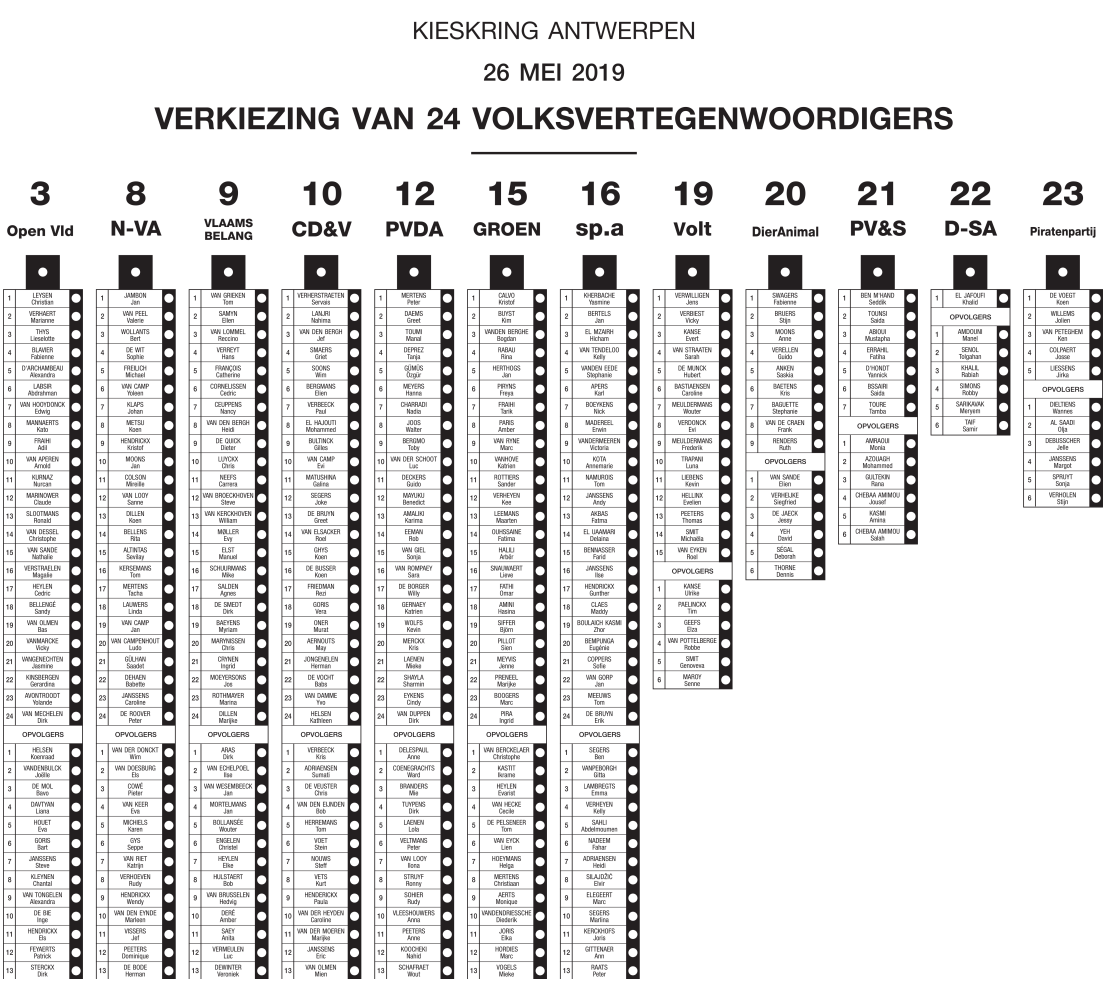


Figure 1: The ballot for the Antwerpen district in the 2019 federal elections.

the highest number of preferential votes or, if it is not enough to break the tie, to the party of the oldest candidate (Art. 168). We do not discuss the fairly complex process that determines how the seats are attributed to the candidates: it is irrelevant to the present paper.

1.0.1. Related work and our contribution

This paper is part of a large body of work deriving assertions for complex social choice functions in a way that allows for and audit in Stark's SHANGRLA framework [Sta20]. Of these, the most closely related work is Blom et. al. [BBR+21], which generates assertions for exact D'Hondt outcomes. The contribution of this paper is to extend those solutions to a setting in which we cannot audit all the assertions necessary for an exact result, instead offering bounds on the seat allocation.

	L_1	L_2	L_3	L_4
# of candidates	3	4	2	3
Party total:	**180**	**96**	**150**	18
$d = 2$	**90**	48	**75**	9
$d = 3$	**60**	32		6
$d = 4$		24		

Merged quotients: **180, 150, 96, 90, 75, 60,** 48, 32, 24.

Figure 2: Illustration of the D'Hondt tallying process with electoral threshold, allocated seats in bold. If there are 6 seats, L_1 gets 3, L_2 gets 1 and L_3 gets 2 seats. L_4 does not reach the electoral threshold of $(180 + 96 + 150 + 18) \times 0.05 = 22.2$, and its quotients are therefore not listed.

1.1. Linked repositories

The RLAUXE project is a general RLA system that incorporates the algorithms described in this report, along with techniques for conducting the audit. The Elections in LEAN repository includes an assertion verifier for relaxed D'Hondt outcomes. It inputs a set of assertions and a set of bounds on the seat allocation of each party, and verifies that the assertions imply the claimed bounds. It also includes implementations of the algorithms in this report. LEAN is used for two main theorems:

- that the relaxation algorithms always produce assertions and bounds that pass verification,
- that if a set of assertions and bounds passes verification, then any tallies satisfying those assertions will, when correctly tallied with D'Hondt, produce seat allocations within the bounds.

Throughout this paper we refer to proofs in the LEAN respository.

2. Auditing the Belgian Federal Elections

2.1. Context of the Audit

Approximately half of the Belgian voters vote on hand-marked and hand-counted paper ballots. Voting happens on a Sunday morning, and counting offices operate on Sunday afternoon and sometimes evening. Each counting office has 6 members, who may tally up to 2400 ballots. The ballots are sorted by parties, then counted, then placed into sealed envelopes. All the ballots in any single envelope must express votes for the same party.

The other half of the voters vote on ballot marking devices that print paper ballots, which are scanned, and counted from the scanned electronic records. Our central focus here is to evaluate the feasibility of running a risk limiting audit for such elections. The audit would be based on an evolution of the current BMDs that has been proposed by

a consortium of universities at the request of the Belgian federal ministry of interior, which is in charge of selecting voting systems that can be used in Belgium. The goal of the audit is to limit the risk that parties are granted the wrong number of seats. As we will see, the election margins tend to be very tight in Belgium, and auditing the seat allocation to the candidate is expected to make audits impractical – this deserves further explorations.

2.2. The election manifest

The RLA starts from an election manifest that indicates, among other things, how many ballots have been cast, how many votes each party received, and how many votes are invalid – this last category includes all the votes that do not express support for one and only one party and includes the blank votes.

The manifest typically contains other book keeping information, including a list of boxes and envelopes and the reported content of each envelope. In the context of hand-counted ballots, each envelope contains ballots supporting a single party, and invalid ballots are also gathered in one envelope. In the context of e-voting, each envelope contains ballots of which an interpretation has been reported during the scanning process. The important observation here is that both the hand marked and the machine marked ballots are stored in a way that offers a commitment on the content of every ballot in every envelope, which can support efficient ballot-level comparison audits. Yes, though it might be a bit fiddly to find the n-th card in the envelope. Are they marked in any way? VT

2.3. Auditing D'Hondt Elections

Let L be a party list. We introduce the following notations and definitions.

- S is the number of available seats.
- $d(i)$ is the divisor for the i-th index. For D'Hondt, $d(i) = i$.
- $\mathcal{P}$ is the set of parties.
- If $P \in \mathcal{P}$ is a party, $\|P\|$ is the number of candidates fielded by P.
- $s :: \mathcal{P} \to \mathbb{N} \cup \{0\}$ is an allocation of seats to parties.

  $s(P)$ is both the number of seats allocated to party P and (if non-zero) the index of the candidate in party P who is reported to get seated with the lowest quotient. This must satisfy $s(P) \leq \|P\|$.
- The *tally* $\mathcal{T}(P)$ is the total number of valid votes for P.
- The *quotient* of the i-th ranked candidate of a party P, denoted $\mathcal{Q}(L, i)$, is the list's tally divided by $d(i)$.

2.3.1. D'Hondt assertions

Let A and B be two party lists in the election.

Notation Define the generalised D'Hondt assertion as

$$DH_{A,B}(w,l) \iff \mathcal{Q}(A,w) > \mathcal{Q}(B,l).$$

That is,

$$DH_{A,B}(w,l) \iff \frac{\mathcal{T}(A)}{d(w)} > \frac{\mathcal{T}(B)}{d(l)}.$$

Usually we check only that the last winner of party A won over the first loser of party B. This is vacuously true if none of A's candidates, or all B's candidates, got seats. The following shorthand captures this case.

$$DH_{A,B} = \begin{cases} DH_{A,B}(s(A), s(B)+1), & \text{if } s(A) > 0 \text{ and } s(B) < \|B\| \\ \text{TRUE}, & \text{otherwise.} \end{cases}$$

We had FALSE for the case where $s(A) = 0$ but I think it should be true. Maybe it doesn't matter because we never use this. Or maybe we never use $DH_{A,B}$ anyway. VT

Note that this assertion is not symmetric (nor antisymmetric): $DH_{A,B}$ does not imply $DH_{B,A}$ (nor $\neg DH_{B,A}$) and, depending on the specifics of an election, these two assertions may both need to be checked. For example, we may need to check both "Party A's 2nd candidate beats Party B's 4th candidate," and also "Party B's 3rd candidate beats Party A's 3rd candidate" which would be part of proving that A got 2 seats and B got 3.

From now on we will ignore the cases in which $s(A) = 0$ or $s(B) = \|B\|$, in which $DH_{A,B}$ does not need to be audited,[2] and consider how to audit this assertion when $s(A) > 0$ and $s(B) < \|B\|$.

For an ordinary D'Hondt election with a set $\mathcal{B}$ of ballots, the reported outcome would be correct if the expected number of seats had been granted and if $DH_{A,B}$ was true for every ordered pair of distinct parties A and B.

When one D'Hondt assertion entails another One DH assertion can imply another via a simple, single form of implication which we call "entailment", meaning it is an assertion about the same two parties that bounds $\mathcal{T}(Q)/\mathcal{T}(P)$ at least as tightly.

Definition 1. $DH_{A,B}(w_1,l_1)$ entails $DH_{A,B}(w_2,l_2)$ *if* $w_1/l_1 \geq w_2/l_2$.

The implication follows immediately from the definition of $DH_{A,B}(w,l)$. Useful special cases are when $w_1 = w_2$ and $l_1 < l_2$ (if w_1 defeats some quotient, it also defeats lower quotients of the same party), or alternatively when $l_1 = l_2$ and $w_2 < w_1$ (if if l_1 is defeated by some quotient, it is also defeated by higher quotients of the same party).

LEAN: This is exactly impliesDHOne in Models/AssertionSet.lean.

An assertion will be tested in the RLA if either it is explicitly present in the assertion list, or it is entailed by an assertion that is present.

[2] We may need to audit other assertions in order to confirm that $s(A) = 0$ or $s(B) = \|B\|$.

2.4. Adding a threshold

In Belgian federal elections, parties can obtain seats only if they obtained at least 5% of the valid votes. We use the following notations for capturing this threshold mechanism. $\Sigma_{P \in \cup \mathcal{P}} \mathcal{T}(P)$ is the *number of valid votes.*

Definition 2 (Threshold)**.** *The* electoral threshold *is*

$$\theta = 0.05$$

We write the assertion that the party P is above the threshold [3] as

$$AT(P) \iff \mathcal{T}(P) \geq \theta \times \Sigma_{Q \in \cup \mathcal{P}} \mathcal{T}(Q)$$

Similarly, $BT(P)$ is the assertion that the party P is below the threshold, that is $BT(P) = \neg AT(P)$. Again, we use these notations both as names for the assertions and for the logical statement that the assertion is true.

Threshold assertions may also be entailed by other assertions. If we have $AT(B)$ and $DH_{A,B}(w, l)$ with $w \geq l$ then $AT(A)$ is implied. The algorithms in this report do not remove these assertions, but the LEAN Verifier reports them as redundant.

2.5. Summary: checking the election outcome

Proposition 1. *In a Belgian federal election, we define $\mathcal{W}$ the set of parties that have at least one reported winner and $\mathcal{L}$ the set of parties that have at least one reported loser, that is $s(L) < \|L\|$. (Some parties may be in both $\mathcal{W}$ and $\mathcal{L}$.) The reported outcome is correct if we prove:*

$$AT(A) \qquad \text{for all } A \in \mathcal{W} \tag{1}$$

$$BT(B) \bigvee \left(\bigwedge_{A \in \mathcal{W} \setminus B} DH_{A,B} \right) \qquad \text{for all } B \in \mathcal{L}. \tag{2}$$

There are also basic validity conditions:

- $\sum_{P \in \mathcal{P}} s(P) = \max(S, \sum_{P \in AT} \|P\|)$ and
- $\forall P \in \mathcal{P}, s(P) \leq \|P\|$.

These are not assertions that need to be audited, but validity conditions to be checked during assertion validation.

Proof. This matches the DHondt.complete data structure in LEAN. We have AssertionSet.complete_assertion_claim_implies_IsStrictDHondt and AssertionSet.complete_assertion_claim_implies_IsDHondt.

Theorem DHondt.eq_doDhondtAllocationWithThreshold_of_isStrictDHondtWithThreshold implies that this allocation is equal to the allocation via the DHondt algorithm. □

VT

The human-readable proof here suffers a bit from not having a precise specification of what the algorithm is.

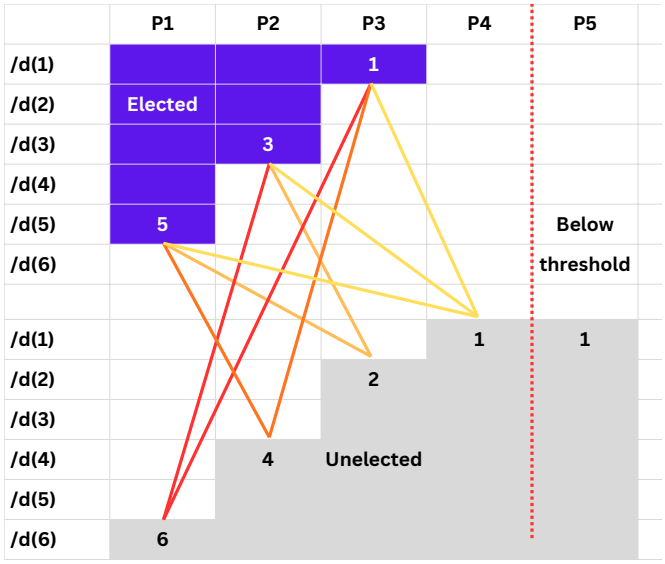


Figure 3: Illustration of D'Hondt and threshold assertions. In some elections, parties that are below the threshold may be D'Hondt winners.

Figure 3 shows an example of the assertions described in Proposition 1.

2.5.1. Assigning all seats

If the reported outcome does not assign all the seats, then the result is correct iff, for every party P,

$$s(P) = \begin{cases} \|P\| \text{ if } AT(P), \\ 0, \text{ otherwise.} \end{cases}$$

It would help to explain this case in the intro—I'm not quite sure what the rules are here. In other words, non-assigned seats can only happen when the losing candidates are coming from parties that did not meet the threshold. Furthermore, every list meeting the threshold must have all of its candidates winning. VT

In this case Proposition 1 reduces to:

1. For all $P \in \mathcal{L}$: $BT(P)$

[3]The threshold only has to be equalled, but our assertions are by convention strict. This does not make any difference for auditing, since ties cannot be audited without a full count anyway.

2. For all $P \notin \mathcal{L}$: $AT(P)$.

VT

Add the point about a conjunction of DH losses instead.

No, I think (if I understand the rules correctly, which quite possibly I don't) that if there are more seats than candidates whose parties get over the threshold, then everyone over the threshold gets all their candidates seated, and that's it.

AE don't we still need to prove DH assertions in this case?

VT

3. Which assertions should be checked?

3.1. Complete assertion sets

When some seats are unassigned, the verification process is clear: we need to check all the AT and BT assertions, one per party. This case is simple and does not happen in practice.

When all the seats are assigned, Proposition 1 can give us up to $2^{|\mathcal{L}|}$ "useful" sets of assertions to check: for each $B \in \mathcal{L}$, it may be that only one of $BT(B)$ or $\wedge_{A \in \mathcal{W}} DH_{A,B}$ is true, or it may be that we can choose either.

The best way to choose is probably to estimate the sample size and choose the option with the lower estimate. Appendix B contains some possible heuristics, and some discussion of using alternative processes more like AWAIRE.

VT

From John C: Using the smallest margin won't give you the highest difficulty solving, one needs to use noerror, which takes into account the upper limit of the assorter, which is not 1 for all the assorters (DH, AT, BT). $noerror = 1.0/(2.0 - assorterMargin/assorter.upperBound())$ see details at https://github.com/JohnLCaron/rlauxe/blob/main/docs/Dhondt.md#31-check-margins

For now, we simply assume that we have some way of choosing which of $BT(B)$ or $\wedge_{A \in \mathcal{W}} DH_{A,B}$ is likely to have a smaller sample size.

3.2. Relaxed assertion sets

Sometimes a single assertion is so hard to prove (because the margin is so close) that auditing is infeasible. In these cases, we may not be able to verify the exact assignment of all seats to all parties.

VT

TODO: I'm leaving out the case in which not all seats are assigned, because I think it's straightforward, being based entirely on thresholds: just audit the thresholds you can, and admit that you can't audit the others.

3.2.1. Motivating example

Consider the outcome of the circonscription of Hainaut, 2024. Apparently, Ecolo fails the threshold. However, the margin by which this occurs is so tiny as to be infeasible to audit. The results are shown in Table 1, with assigned seats in blue.

We'll have to fix this up when we correctly calculate the threshold as a function of the total, not the valid ones - TODO. Actually our intro says it's a function of the valid ones. TODO check. Ans: it's 5% of *valid* votes, which does *not* include blanks.

VT

	MR	PS	LES ENGAGÉS	PTB	N-VA	DéFI	ECOLO	BELG. UNIE-BUB	COLLECTIF CITOYEN	LUTTE OUVRIERE	CHEZ NOUS	BLANCO
Seats	5	6	3	3	0	0	0	0	0	0	0	0
%	26	29	15	14	1.9	2.1	5	0.33	0.97	0.63	3.0	1.8
Votes	192759	213501	114559	103339	14184	15189	36750	2426	7201	4680	22039	13224
/2	96380	106751	57280	51670	7092	7595	18375	1213	3601	2340	11020	6612
/3	64253	71167	38186	34446	4728	5063	12250	809	2400	1560	7346	4408
/4	48190	53375	28640	25835	3546	3797	9188	607	1800.25	1170	5510	3306
/5	38552	42700	22912	20668	2837	3038	7350	485	1440.2	936	4408	2645
/6	32127	35584	19093	17223	2364	2531.5	6125	404	1200	780	3673	2204
/7	27537	30500	16366	14763	2026	2170	5250	347	1029	669	3148	1889

Table 1: Election outcome for Hainaut, 2024, in which 17 seats were allocated. Seated quotients are shown in blue. The threshold, at 5%, is 36993, which the Ecolo party very narrowly falls short of.

Auditing the whole election outcome is infeasible because $BT(\text{Ecolo})$ is infeasible. Ecolo's first candidate is not a D'Hondt loser, so that alternative audit option does not work. If we simply omit the assertion $BT(\text{Ecolo})$ from the equations in Proposition 1, then we leave open the possibility that Ecolo might have won far more than 1 seat. If it is true that actually $AT(\text{Ecolo})$, then (apparently) PTB would miss out on its 3rd seat, which is the last allocated. We cannot therefore hope to verify that PTB got 3 seats without verifying $BT(\text{Ecolo})$. The best we can hope for is to verify

- that PTB got 2 or 3 seats,
- that Ecolo got 0 or 1 seat,
- that all other parties got exactly their apparent allocation.

That is, either Ecolo passed the threshold and got a seat, and PTB got 2 seats, or Ecolo failed the threshold and got no seats, and PTB got 3 seats.

In this example, Ecolo would get only one seat if it crossed the threshold, though this is not true in general. It is always true that the last seat allocated (i.e. the smallest number) is the one that is lost.

3.2.2. Verifying relaxed assertion sets

The following proposition gives sufficient conditions for a relaxed assertion set to imply a claimed set of seat bounds. Informally, we need to check that all apparently-certain winners definitely win, and that all apparently-certain losers definitely lose. We do not need to check anything explicitly about the uncertain quotients between lo_P and hi_P.

Proposition 2. *Consider an election with S seats and party set $\mathcal{P}$, a collection $\mathcal{A}$ of AT, BT and DH assertions, and claimed seat bounds (lo_P, hi_P) for every $P \in \mathcal{P}$. pull out the def of entailed; write elsewhere.* VT

Suppose that the following basic consistency properties hold.[4]

[4]Technically, our proof assumes that every party not in BT has a tally strictly greater than zero, but

1. *No party has both an AT and a BT assertion.*
2. $\sum_{P\in\mathcal{P}} lo_P \le S$.
3. $\mathcal{P}$ *contains every party standing in the contest.*
4. $0 \le lo_P \le hi_P \le \min(S, \|P\|)$ *for every* P.
5. $\sum_P hi_P \ge S$. *We will deal with incomplete seat allocation later.* VT

Write

$$\textit{possibly-AT} = \{\, Q \in \mathcal{P} : BT(Q) \notin \mathcal{A} \,\},$$
$$\textit{certainly-AT} = \{\, Q \in \mathcal{P} : AT(Q) \in \mathcal{A} \,\}.$$

The following are then sufficient to verify the claimed bounds. That is, the assertion set $\mathcal{A}$ *implies that every D'Hondt allocation* s *of* S *seats among the parties in* $\mathcal{P}$ *(under the true threshold decisions) satisfies* $lo_P \le s(P) \le hi_P$ *for all* $P \in \mathcal{P}$.

1. (Lower bounds) *For every* P *with* $lo_P > 0$*:*
   a) $AT(P) \in \mathcal{A}$, *and*
   b) *for every* $Q \in$ *possibly-AT* $\setminus \{P\}$ *there is some* l_Q *with* $1 \le l_Q \le \|Q\| + 1$ *s.t.* $(DH_{P,Q}(lo_P, l_Q)$ *is entailed or* $l_Q = \|Q\| + 1)$, *and*
   c) $lo_P + \sum_{Q\in\textit{possibly-AT}\setminus\{P\}}(l_Q - 1) \le S$.
2. (Upper bounds) *For every* P, *either:*
   a) $BT(P) \in \mathcal{A}$, *or*
   b) $hi_P = \|P\|$, *or*
   c) *for every* $Q \in$ *certainly-AT* $\setminus \{P\}$ *there is some* w_Q *with* $0 \le w_Q \le \|Q\|$ *s.t.* $(DH_{Q,P}(w_Q, hi_P + 1)$ *is entailed or* $w_Q = 0)$, *and*
   d) $hi_P + \sum_{Q\in\textit{certainly-AT}\setminus\{P\}} w_Q \ge S$.

VT

TODO: Should be altered slightly for the case that not all S seats are allocated.

Proof. Lower bound. We wish to show that at most $S-1$ quotients from above-threshold parties can defeat or tie with P's lo_P-th quotient. Then, if P is above the threshold, that quotient must win.

P's lo_P-th quotient exists by validity condition 4. The quotients that can rank at or above P's lo_P-th are its own first $\mathrm{lo}_P - 1$, and, for each other party Q that might

since the assertion verifier does not see tallies, this is not part of the verification process—we simply assume that someone would notice if the election's total vote was zero. The LEAN proof does not use this assumption, instead constructing a validity proof by contradiction that is different from the proof given below.

be above the threshold, ($Q \in$ possibly-AT $\setminus \{P\}$), at most its first $l_Q - 1$: either Q's l_Q-th quotient loses to P's lo_P-th, or $l_Q = \|Q\| + 1$ and Q has only $l_Q - 1$ quotients. Parties with a BT assertion receive no seats. So at most $\text{lo}_P - 1 + \sum_Q (l_Q - 1) \leq S - 1$ quotients are greater than or equal to P's lo_P-th, and it is seated, given that P is above the threshold.

Upper bound. If $BT(P)$ holds or $\mathcal{T}(P) = 0$ then $s(P) = 0$.[5] So assume $\mathcal{T}(P) > 0$.

If $\text{hi}_P = \|P\|$, then P has no $\text{hi}_P + 1$-th quotient, so $s(P) \leq \text{hi}_P$.

We show that at least S quotients of above-threshold parties are strictly greater than P's $(\text{hi}_P + 1)$-th quotient; hence that quotient must lose.

First, P's own first hi_P quotients are strictly greater than its $(\text{hi}_P + 1)$-th, since $\mathcal{T}(P) > 0$. If P is below the threshold it gets no seats and trivially satisfies the upper bound, so we may take P to be above it.

Second, each $Q \in$ certainly-AT $\setminus \{P\}$ is above the threshold, and if $\|Q\| \geq w_Q > 0$ then $DH_{Q,P}(w_Q, \text{hi}_P + 1)$ holds, so Q's first w_Q quotients exist and are strictly greater than P's $(\text{hi}_P + 1)$-th. If $w_Q = 0$ then it contributes nothing to the sum. Therefore there are at least

$$\text{hi}_P + \sum_{Q \in \text{certainly-AT} \setminus \{P\}} w_Q \;\geq\; S$$

quotients of above-threshold parties strictly greater than P's $(\text{hi}_P + 1)$-th. □

Proof (LEAN). This is the result proven in VerifiedVerifier.check_sound, for D'Hondt with all party sizes at least S. The details of the proofs are different, but the result is the same for that case. The LEAN code deals differently with the validity conditions: conditions 1 and 2 are checked explicitly; condition 3 is a hypothesis, and the last two conditions are absent. TODO: this description is from Claude - check, and revise when we add the party sizes. VT □

4. Deriving relaxed assertion sets

In this section we devise assertions for proving part of an election outcome, for use when proving the entire election outcome correct is infeasible. Because the number of seats is fixed, an uncertainty about one seat usually implies (the opposite) uncertainty about at least one other.

4.1. Equivocating over whether a party failed the threshold

Suppose we have a party R that is too close to the threshold to verify. We wish to relax the conditions in Proposition 1 so that everything else is verified.

When a party R has narrowly failed the threshold, and we wish to relax the assertions to allow that it might have passed (or vice versa), then we still need to prove that all the (other) apparently-winning parties passed the threshold and all the apparent losers

[5] Assuming the total vote is not zero.

(other than R) lost. We also need to think carefully about which DH assertions are required.

Define the following subsets of $\mathcal{P}$:

AT are those (clearly, verifiably) *above* the threshold—we include an AT assertion for each party in this set;

BT are those (clearly, verifiably) *below* the threshold.

Proposition 3. *In the same style as Proposition 1, we have two options for implying that a party L got no seats: either $BT(L)$ or*

$$\bigwedge_{W \in AT} DH_{W,L}(s_W, 1) \qquad s.\ t. \quad \sum_{W \in AT} s_W \geq S.$$

Proof. TODO, but hopefully obvious. (**ref LEAN?) □

Depending on the data, one or the other, or neither or both, may be feasible for audit.

Note that s_W in the above expression need not be the apparent seat allocations to AT parties—it suffices to check that there are enough clear DH winners among the AT parties to defeat L. Then we should use a different notation from s. Note also that it is *not* sufficient to use parties that appear to be above the threshold but for which an AT assertion is not included (because it is too hard to verify). VT

We define NS ("No seats") to be the set of parties for which we can feasibly audit that the party got no seats, by either of the options in Proposition 3.

NS are the parties for which there are sufficient feasible assertions to show the party got no seats, either via BT or by the set of DH loss assertions given in Proposition 3. For these parties, if both options are available, we include whichever assertion (set) is easier.

The last set are those for which there is uncertainty—parties that may or may not get any seats.

UT are those parties in neither AT nor NS.

The discussion of which ones we decide are "too close" is a separate question. Again we simply assume that we have some way of estimating sample sizes, and some judgement about which ones are acceptable. In this section, we simply specify how we state the relaxation, and which assertions will imply a given relaxation.

Useful logical fact If X, Y, Z are logical propositions, the following is true:

$$Y \wedge Z \implies (X \wedge Y) \vee (\neg X \wedge Z)$$

We will set X to be the assertion $AT(R)$ and let Y and Z respectively be all the other things we would need to check when $AT(R)$ is believed true (resp false). In this way,

we prove that either ($AT(L)$ true and everything else is correct) or ($AT(R)$ is false and everything else is correct). This same idea generalises to multiple simultaneous uncertain propositions:

If $X_1, X_2, \ldots, X_n$ are logical propositions and $t \in \{0,1\}^n$, write $\vec{X}_t$ to be the conjunction of the logical propositions, with X_i negated iff $t_i = 1$.

Definition 3. *If $X_1, X_2, \ldots, X_n$ are logical propositions and $t \in \{0,1\}^n$, write* $\textsc{all}(X \leftarrow t)$ *to be the conjunction of those propositions, but with X_i negated if $t_i = 1$. That is,*

$$\textsc{all}(X \leftarrow t) = \bigwedge_{i \in [1,n]} Z_t, \text{ where } Z_i = \begin{cases} X_i, & \text{if } t_i = 0, \\ \neg X_i & \text{if } t_i = 1. \end{cases}$$

Useful logical fact 2 If $X_1, X_2, \ldots, X_n$ are logical propositions and $t \in \{0,1\}^n$, the following generalisation of logical fact 1 is true:

$$\bigwedge_{t \in \{0,1\}^n} Y_t \implies \bigvee_{t \in \{0,1\}^n} (\textsc{all}(X \leftarrow t) \wedge Y_t)$$

Proof: one of the assignments τ of negations to $\vec{X}$ makes the whole conjunction true (it's just the one that has a 1 in each index i for which X_i is false). We're given Y_t is true for all t, so Y_τ is true, hence $\textsc{all}(X \leftarrow \tau) \wedge Y_\tau$ is true, so the whole disjunction on the RHS is true. VT

Make the link between the logical prop'n and the assertion gathering in subsubsection 4.1.2 clearer.

4.1.1. Algorithm for relaxing threshold assertions

This leads us to the following simple algorithm for dealing with uncertainty about thresholds. This algorithm is not implemented, but is included here to help the reader understand the main ideas.

4.1.2. Algorithm for threshold uncertainty

1. Initialise the assertion set with $AT(P)$ assertions for all $P \in AT$ and, similarly, either $BT(P)$ or $\bigwedge_{W \in AT} DH_{W,L}(s_W, 1)$ s. t. $\sum_{W \in AT} s_W \geq S$ for all $P \in NS$.

2. For all $2^{|UT|}$ possible assignments of AT or BT to the parties in UT, calculate the DH assertions from Proposition 1 and add the DH assertions to the assertion set. (Do not include AT or BT assertions for parties in UT.)

3. For each party P, its maximum and minimum seat counts are the maximum and minimum values encountered at any point in Step 2.

4. Return the seat count bounds and the assertion set.

Why is it valid to simply take the union of all the complete assertion sets from every possible assignment of threshold-passing booleans to UT? This is implied by useful logical fact 2. TODO - lay out more carefully; correct repeat of Y_t with different meanings. VT

This algorithm is correct, but we will improve upon it in two ways. First, we need to deal with the case that some DH assertions are infeasible—this is in the next section.

Second, we can show that it is not necessary to check all $2^{|UT|}$ possible assignments, but only the cases in which

1. all $P \in UT$ are above the threshold,
2. all $P \in UT$ are below the threshold,
3. for each $P \in UT$, P is above the threshold and all other parties in UT are below.

This is proven in LEAN in RelaxThresholds.lean. There's a proof in the LEAN repo but it's difficult for humans to read. Unclear whether we care or we should just do the $2^{|UT|}$ thing as John has been doing for a while. VT

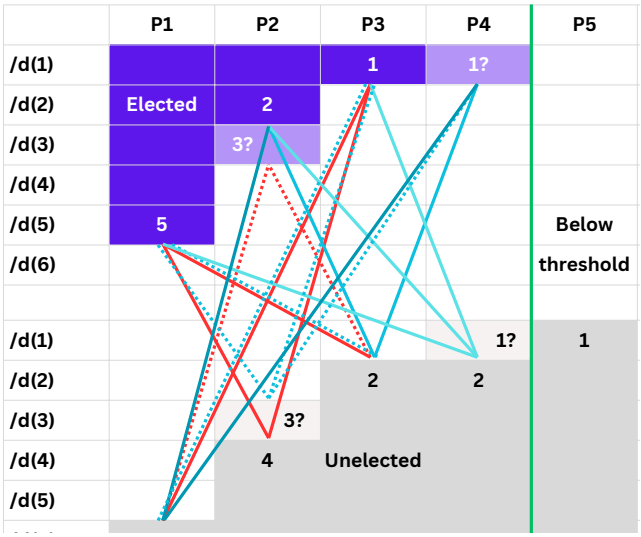


Figure 4: Illustration of the assertions to use when it is infeasible to audit whether a particular party (in this case P4) is above or below the threshold. The example shows what happens when either (P4 is above the threshold and P2 gets only 2 seats) or (P4 is below the threshold and P2 gets 3 seats). Blue lines are relevant to the first disjunct, red lines to the second. Redundant assertions are dotted.

Appendix **??** contains the exact assertion sets and some simplifications. Figure 4 shows an example. This is simpler than it looks because many assertions are either repeated, or logically implied by others and hence redundant.

4.2. Relaxing in the case that a DH assertion is infeasible

When there is an infeasible DH assertion $DH_{A,B}(w, l)$, we can omit it and generate the assertions that allow the seat to be assigned to either side of the infeasible DH comparison. The assertions are given in **??** in Appendix **??**. This means adding an assertion that the second-lowest winner of Party A still defeated B's highest loser, and that the lowest winner of Party A defeated B's second-highest loser. Figure 5 shows an example—the added assertions are shown in blue.

(Note there is no guarantee that the lowest-margin DH assertion impacts either the overall highest loser or lowest winner.)

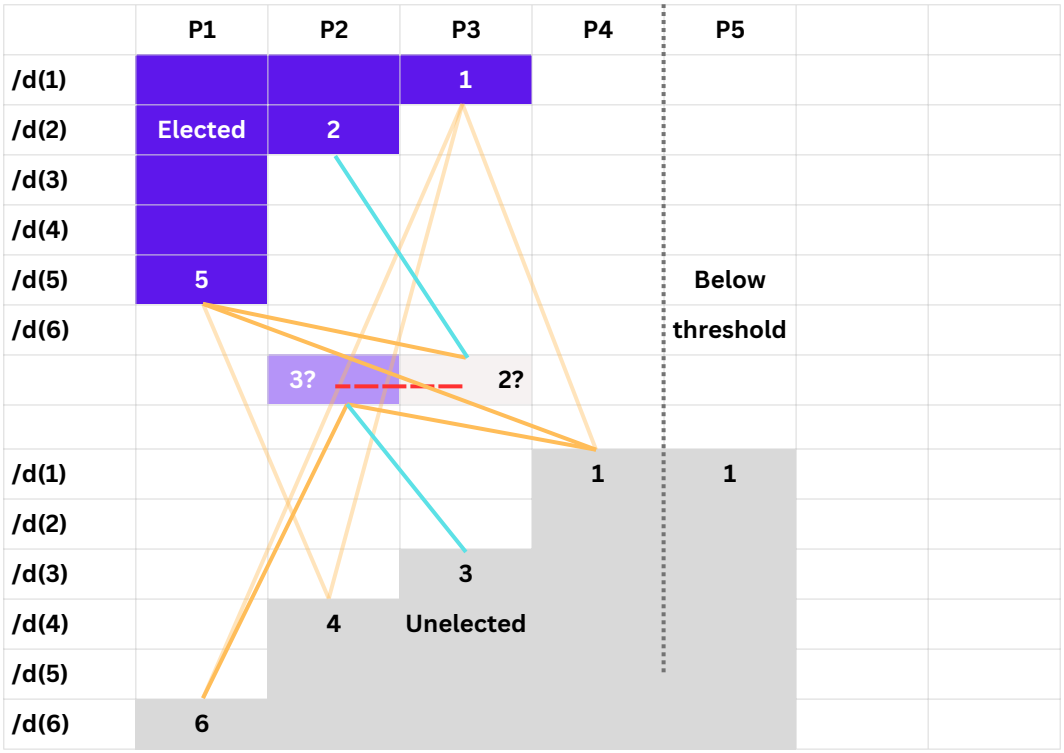


Figure 5: Illustration of the assertions to use when it is infeasible to audit a DH assertion. There are 9 seats available. The example shows what happens when the DH assertion that P2's 3rd candidate defeats P3's 2nd candidate is relaxed—this assertion (which is removed) is shown as a dashed red line. Blue lines show the relaxed DH assertions that replace it. The other assertions stay the same—these are shown in orange. We leave P2's 3rd candidate and P3's second candidate in an "unsure" category. Note there does not have to be an assertion between P1's 5th candidate and P3's 3rd candidate, because this holds transitively. Similarly P2's 2nd vs P3's 3rd.

This process can be iterated if there are multiple infeasible DH assertions.

4.2.1. Relaxing multiple DH comparisons

DH winner-loser pairs can be relaxed iteratively, by maintaining the following indexes for each party:

DH$_W$ the index of the lowest quotient that is a certain winner,

DH$_{\mathsf{UL}}$ the index of the highest quotient that is not a certain loser.

With no relaxations, assuming we make DH assertions only for apparent DH winners, $DH_W(p) = DH_{UL}(p) = s(p)$ for all $p \in \mathcal{P}$. In the example in Figure 5, $DH_W(P2) = 2, DH_{UL}(P2) = 3, DH_W(P3) = 1$ and $DH_{UL}(P3) = 2$. Party A's first clear loser is $DH_{UL}(A)+1$. Its first uncertain candidate, if it exists, is $DH_W(A)+1$. Its last uncertain candidate, if it exists, is $DH_{UL}(A)$.

This suggests the following algorithm (ignoring thresholds).

4.2.2. DH-relaxing algorithm

1. Generate assertions for the apparent outcome, as in Proposition 1.
2. Calculate sample sizes for the DH assertions. If all are acceptable, RETURN those assertions and the exact seat allocation.
3. Initialize $DH_W(p) = DH_{UL}(p) = s(p)$ for all $p \in \mathcal{P}$.
4. For all unacceptable *DH* assertions $DH_{A,B}(w, l)$ in the current assertion set:
   a) Remove $DH_{A,B}(w, l)$ from the assertion set,
   b) If $w > 1$, add $DH_{A,B}(w-1, l)$ to the assertion set,
   c) Set $DH_W(A) = \min(w-1, DH_W(A))$. This moves A's w-th quotient from the winning set into the uncertain-win set if it is not there already.
   d) If $l < \|B\|$ (i.e. if B has any other losers), add $DH_{A,B}(w, l+1)$ to the assertion set,
   e) Set $DH_{UL}(B) = \max(l, DH_{UL}(B))$. This moves B's l-th quotient from the losing set into the uncertain set if it was not there already.

   If all are acceptable, RETURN those assertions and the corresponding bounds ($DH_W(P)$ to $DH_{UL}(P)$).
5. Go to Step 4.

This algorithm is in LEAN as RelaxDH.relaxDH. The proof that its output always passes the verifier is RelaxDH.relaxDH_implies_verified.

4.3. Relaxing threshold and DH assertions

The aim is to derive bounds for the number of seats allocated to each party. We choose some extreme scenarios that give worst- and best-case scenarios for parties in certain classes. For all these scenarios, a DH relaxation step can also be done. This provides upper and lower bounds on the number of allocated seats for each party.

The following algorithm combines a shortcut version of 4.1.2 with the DH-relaxing algorithm in 4.2.2. Again, it is correct to use all $2^{|UT|}$ possibilities for above/below threshold values in UT rather than the shortcut shown here.

4.3.1. Complete algorithm

1. Initialise the assertion set with $AT(P)$ assertions for all $P \in AT$ and, similarly, either $BT(P)$ or $\bigwedge_{W \in AT} DH_{W,P}(s_W, 1)$ s. t. $\sum_{W \in AT} s_W \geq S$ for all $P \in NS$.
2. If UT is empty, generate DH assertions for the apparent outcome and run the algorithm from subsubsection 4.2.1 to derive relaxed DH assertions and the corresponding relaxed-DH seat bounds. Add the relaxed DH assertions to the assertion set; RETURN the assertion set and relaxed-DH seat bounds.
3. Assume all $P \in UT$ are *above* the threshold (but do not add AT assertions for them).
   a) Run the algorithm from subsubsection 4.2.1 to derive relaxed DH assertions and the corresponding relaxed-DH seat bounds—call them $DH_W^{\text{low}}(A), DH_{UL}^{\text{low}}(A)$.

   Call the corresponding assertion set the *Low Assertion Set*. Add it to the assertion set.
4. Assume all $P \in UT$ are *below* the threshold (but do not add BT assertions for them).
   a) Run the algorithm from subsubsection 4.2.1 to derive relaxed DH assertions and the corresponding relaxed DH values $DH_W^{\text{high}}(A), DH_{UL}^{\text{high}}(A)$.

   Call the corresponding assertion set the *High Assertion Set*. Add it to the assertion set.
5. For each $P \in UT$, assume that P is above the threshold and all other parties in UT are below it (but do not add any AT or BT assertions).
   a) Run the algorithm from subsubsection 4.2.1 to derive relaxed DH assertions and the corresponding relaxed DH values $DH_W^{\text{P-solo}}(A), DH_{UL}^{\text{P-solo}}(A)$.

   Call the corresponding assertion set the *P-solo Assertion Set*. Add it to the assertion set.
6. The overall seat bounds are, for each party, the maximum and minimum seats allocated in any of the above steps.

This algorithm is in LEAN as RelaxDHAndThresholds.relaxAll. The proof that its output always passes the verifier is RelaxDHAndThresholds.relaxAll_implies_verified.

4.3.2. Summary/implications

Each party's seats are bounded by the max and min that they achieve through Algorithm 4.3.1. In some cases (described below) it is obvious when that occurs.

If $P \in AT$, then $DH_W^{\text{low}}(P)$ are P's clear winners even under the most pessimistic scenario (for P) of threshold-uncertain parties meeting the threshold, and even after all the DH relaxations. The most optimistic scenario (for P) is that all uncertain parties fail the threshold, then P gets all its DH-uncertain seats in that case.

This gives us the following bounds:

$$\forall P \in AT, DH_W^{\text{low}}(P) \leq s(P) \leq DH_{UL}^{\text{high}}(P).$$

If $P \in UT$, then it could be below the threshold and get 0 seats. Its maximum occurs when it is the sole above-threshold party out of all the uncertain parties, and it gets all its DH-uncertain seats in that case. Hence:

$$\forall P \in UT, 0 \leq s(P) \leq DH_{UL}^{\text{P-solo}}(P).$$

Obviously if $P \in NS$ then $s(P) = 0$.

This gives a complete characterisation of the bounds on every party's true seat allocation.

4.3.3. Summary of Assertions

From subsubsection 4.3.1, we have the union of

- $AT(P)$ assertions for all $P \in AT$ and, similarly, $BT(P)$ or the DH conjunction from Proposition 3 for all $P \in NS$,
- the low assertion set,
- the high assertion set,
- the P-solo assertion set for each $P \in UT$.

Auditors also need to check the basic validity conditions:

- $\sum_{P \in \mathcal{P}} s(P) = \max(S, \sum_{P \in AT} \|P\|)$, and
- $\forall P \in \mathcal{P}, 0 \leq s(P) \leq \|P\|$.

References

[BBR+21] Michelle L. Blom, Jurlind Budurushi, Ronald L. Rivest, Philip B. Stark, Peter J. Stuckey, Vanessa Teague, and Damjan Vukcevic. Assertion-based approaches to auditing complex elections, with application to party-list proportional elections. In *Electronic Voting - 6th International Joint Conference, E-Vote-ID 2021*, volume 12900 of *Lecture Notes in Computer Science*, pages 47–62. Springer, 2021.

[Sta20] Philip B Stark. Sets of half-average nulls generate risk-limiting audits: Shangrla. In *International conference on financial cryptography and data security, Voting workshop*, pages 319–336. Springer, 2020. Arxiv: `https://arxiv.org/pdf/1911.10035`.

A. Defining assorters for DH and AT assertions.

A.1. D'Hondt assorters

Following Blom et al. [BBR+21, Section 5.2], we can define the proto-assorter of a ballot b for this assertion as

$$g_{A,B}(b) = \frac{b_A}{s(A)} - \frac{b_B}{s(B)+1},$$

where b_P is set to 1 if the ballot b supports party P, and b_P is set to 0 otherwise.

We can turn this proto-assorter into an assorter

$$h_{A,B}(b) = b_A \cdot \frac{s(B)+1}{2s(A)} + \frac{1-b_B}{2}.$$

We can verify that $h_{A,B}(b)$ is 0 for a vote for B, is 1/2 for a vote for neither A nor B, and is $\frac{s(B)+1}{2s(A)} + \frac{1}{2}$ for a vote for A. Furthermore, when the tally is computed on a set of ballots $\mathcal{B}$, we will have

$$DH_{A,B} \iff \sum_{b\in\mathcal{B}} h_{A,B}(b)/|\mathcal{B}| > 1/2.$$

A.2. Threshold assorters

Threshold assorters can be use the supermajority formulation from SHANGRLA[Sta20].

$$\textit{above-}\theta_P(b) = \frac{b_P}{2\theta} + \frac{b_{INV}}{2},$$

where b_{INV} is 1 if b is invalid, zero otherwize.

Proposition 4. *above-θ_P, defined by above-$\theta_P(b) = \frac{b_P}{2\theta} + \frac{b_{INV}}{2}$, is a valid assorter for AT. That is,*

$$\sum_{b\in\mathcal{B}} \textit{above-}\theta_P(b)/|\mathcal{B}| > 1/2. \implies AT(P),$$

where $\mathcal{B}$ is the total set of ballots.

Proof. Suppose there are v_P votes for party P, v_O in total for every other party, and v_I invalid. If the assorter mean is greater than 1/2, then

$$\begin{aligned} 1/2 &< \sum_{b\in\mathcal{B}} \textit{above-}\theta_P(b)/|\mathcal{B}| \\ &= (\frac{v_P}{2\theta} + \frac{v_I}{2})/(v_P + v_O + v_I) \\ \therefore v_P + v_O + v_I &< v_P/\theta + v_I \\ \therefore v_P &> \theta(v_O + v_P). \end{aligned}$$

$v_O + v_P$ is the total of valid votes, so $AT(P)$ follows. □

We can verify that *above-*$\theta_P(b)$ is 0 for a vote for some other party, is 1/2 for invalid votes, and is $\frac{b_P}{2\times\theta}$ for a vote for P ($= 10$ in our case).

Similarly, the assorter for $BT(A)$ is:

$$\textit{below-}\theta_A(b) = \frac{1 - b_A - b_{INV}}{2(1-\theta)} + \frac{b_{INV}}{2}$$

This is the opposite of *at*: it is 0 when b is a vote for A. If the valid votes comprise x votes for A and $x(1-\theta)$ votes for other parties, then the total is exactly 1/2.

Proposition 5. *below-*θ_A*, defined by below-*$\theta_A(b) = \frac{1-b_A-b_{INV}}{2(1-\theta)} + \frac{b_{INV}}{2}$*, is a valid assorter for BT. That is,*

$$\sum_{b\in\mathcal{B}} \textit{below-}\theta_A(b)/|\mathcal{B}| > 1/2 \implies BT(A).$$

Proof. Suppose there are v_A votes for party A, v_O in total for every other party, and v_I invalid. If the assorter mean is greater than 1/2, then

$$\begin{aligned} 1/2 &< \sum_{b\in\mathcal{B}} \textit{below-}\theta_A(b)/|\mathcal{B}| \\ &= \left(\frac{v_O}{2(1-\theta)} + \frac{v_I}{2}\right)/(v_A + v_O + v_I) \\ \therefore v_A + v_O + v_I &< v_O/(1-\theta) + v_I \\ \therefore v_A &< \theta(v_O + v_A). \end{aligned}$$

$v_O + v_A$ is the total of valid votes, so $BT(A)$ follows. □

We can verify that *below-*$\theta_A(b)$ is 0 for a vote for A, is 1/2 for invalid votes, and is $\frac{1}{2(1-\theta)}$ for a vote for some other party (≈ 0.53 in our case).

$$BT(A) \iff \sum_{b\in\mathcal{B}} \textit{below-}\theta_A(b)/|\mathcal{B}| > 1/2.$$

Formula from JC, with an extra term for invalid votes. VT

B. Estimating which disjunct requires fewer samples

This section discusses possible heuristics for choosing which of $BT(B)$ or $\wedge_{A\in\mathcal{W}} DH_{A,B}$ to audit.

(we could verify both, but that appears to be a waste of resources). AE unless an AWAIRE style method is used, which takes into account that only one needs to be verified

We discuss some heuristics here. Let us define DH-Loser(B) for $B \in \mathcal{L}$ the assertion that $\wedge_{A\in\mathcal{W}} DH_{A,B}(,.)$

B.1. Check margins

If θ is the electoral threshold, we can compute $m_1 = \theta - \mathcal{Q}(B, s(B)+1)$ and m_2 to be the minimum over all $A \in \mathcal{W}$ of the number of votes that would need to be switched from list A to list B in order to make the first loser of B win over the last winner of A.

For parties $B \in \mathcal{L}$, we have a choice: do we test $BT(B)$ or all $DH_{A,B}$(,f)or every $A \in \mathcal{W}$, i.e., DH-Loser(B). This choice comes down to which one we think are easiest to verify. We can estimate this by using the reported margins of each assorter. Let m_1 be the assorter margin of $BT(B)$:

$$m_1 = \theta \times V - \mathcal{T}(B)$$

Let m_2 be the assorter margin of the D'Hondt assertions:

$$m_2 = \min_{A \in \mathcal{W}} \mathcal{Q}(A, s(A)) - \mathcal{Q}(B, s(B)+1)$$

Then, if $m_1 > m_2$, we test $BT(B)$ and we test DH-Loser(B) otherwise. If $m_1 < 0$ B was reported below the threshold. However, m_2 cannot be below 0 (in that case, reported result does not match CVRs) and is 0 when there is a reported tie.

AE Is there something missing here? I'll try and restate it below (if I understood correctly)

Question: Not sure whether this may not be optimal, especially when there are many parties, because DH-Loser(B) is big conjunction of assertions and, even if each assertions is easier to verify than $BT(B)$, we need to check many of them. On the other hand, we check them all in parallel, so the expected number of assertions may not change, but just the standard deviation around that expectation?

True. The computational cost probably doesn't matter, because at worst it's $O(p^2)$ where p is the number of parties. However, the expected sample size of a big conjunction is indeed higher than the expected sample size of the worst thing, so (like RAIRE) this is an approximation, and I agree that in this case it might not be a very good one. That is, if we had a BT assertion with margin μ, and a collection of lots of DH assertions with margin $\mu + \epsilon$, we're saying at the moment that we should take the DH option, but actually it's probably better to take the BT option. Actually now I see someone has sketched this below.

AE Computationally, there is a higher cost of testing $DHondtLoser$ as there are more assertions to test. However, you're posing an interesting question with regards to statistical efficiency. The number of assertions should not depend on the standard deviation of the running sample, but maybe there are some robustness considerations here with respect to expected

B.2. Compare expectations based on ALPHA martingales

We can write assertions for $BT(B)$ and DH-Loser(B) and estimate how many ballots we would need to sample for verifying each of these two assertions. The we decide to pick the one that offers the smallest expectation on the number of ballots to sample.

Question: Is this any better than relying on the margins as before? Yes, I think if we could do this carefully then it's probably better. Could be done either on-the-fly AWAIRE style, or by simulation in advance. However, I'd be surprised if it makes a huge difference because the bad cases will probably be dominated by one bad assertion anyway.

Furthermore, are such local estimations good estimates or what will happen when they are composed? More precisely, if C is (a set of) assertions, is it the case that if Assertion A is cheaper to verify than Assertion B, then verifying A and C is cheaper than verifying B and C? I would conjecture that this conjecture is correct in the present case because all the assertions are related to a single race, so everything is checked in parallel and, in the worst case, the cost of verifying C will dominate everything?

Sorry don't understand - why is it the disjoint sum rather than set union?

AE Just realised something... For $DHondtLoser(1$ we estimate how many ballots for B we need for each $DHondtA, B$ and how many ***mutually exclusive*** ballots for each A we need. Then we can estimate overall sampling effort for $DHondtLoser(1$ by taking the maximum ballots needed for B and sum the number of ballots needed for every A. This should be more robust than just taking maximum margin of each $DHondt$ individually

VT

AE Not necessarily, AWAIRE is useful when CVRs are available but perhaps slightly inaccurate. And even

B.3. Compare expectations based on RAIRE (or) AWAIRE)

I understand that AWAIRE would remove the need to decide in advance which set of assertion to check and would rather choose adaptively based on the ballots that are sampled. But this is only useful when CVRs are not available, which is not the case in Belgium.

Question: Does RAIRE bring any benefit over ALPHA in the D'Hondt case here? I have no idea of how hard it would be to adapt it to the D'Hondt setting.

I'd reserve the term RAIRE for IRV, and say "choosing the assertions that maximin margin in advance" but agree with AEK - either should work. maximin is simpler; AWAIRE is better if you have enough errors that the optimal assertions are actually different from what the CVRs told you would be optimal. We could also do a more careful job of estimating the sample size of those conjunctions in advance—agree that this would be better, and that AWAIRE might do it automatically (but it might still be better to do it in advance). OTOH if there's only one small-margin assertion then that will dominate, so the extra complexity might not be worth it.